

TradeVision AI — User Manual

Version: 1.0 **Project:** TradeVision AI — Rule-based algorithmic trading & research console **Audience:** Traders, analysts, and system operators using the local development deployment at `http://localhost:5173` (Vite) or `http://localhost` (nginx).

This manual documents **every menu, page, and feature** of the application as it currently runs in the local development stack. It is a reference for *using* the UI, not an API specification for external consumers (API details are summarized in the appendices for completeness).

Table of Contents

1. [Overview](#)
2. [Getting Started](#)
 - [2.1 What runs locally](#)
 - [2.2 Ports & services](#)
 - [2.3 Credentials](#)
 - [2.4 First login](#)
3. [Navigation structure](#)
4. [Authentication & user management](#)
 - [4.1 Logging in](#)
 - [4.2 Password & session](#)
 - [4.3 API keys](#)
 - [4.4 Roles & permissions](#)
5. [Dashboard](#)
 - [5.1 Home](#)
 - [5.2 Portfolio](#)
 - [5.3 Positions](#)

- [5.4 Orders](#)
 - [5.5 Trades](#)
 - 6. [Analytics & Risk](#)
 - [6.1 PnL analytics](#)
 - [6.2 Analytics index redirect](#)
 - 7. [Risk decisions](#)
 - 8. [Research / Backtesting](#)
 - [8.1 Backtests](#)
 - [8.2 Edge Validation](#)
 - [8.3 Cost Sensitivity](#)
 - [8.4 Walk-Forward](#)
 - 9. [Rules & Signals](#)
 - [9.1 Rule configs](#)
 - [9.2 Rule executions](#)
 - [9.3 Signals](#)
 - 10. [Recommendations](#)
 - 11. [Memory & Watchlist](#)
 - [11.1 Trader Memory](#)
 - [11.2 Watchlist](#)
 - 12. [System & Observability](#)
 - [12.1 Journal](#)
 - [12.2 Audit log](#)
 - [12.3 Pipeline health](#)
 - [12.4 Kill switch](#)
 - [12.5 Reconciliation](#)
 - [12.6 Raw events](#)
 - [12.7 News](#)
 - [12.8 Patterns](#)
 - 13. [UI features](#)
 - [13.1 Dark / light mode](#)
 - [13.2 Data states](#)
 - 14. [Configuration reference](#)
 - [14.1 Environment variables \(compose\)](#)
 - [14.2 Django settings overrides](#)
 - 15. [API reference \(summary\)](#)
 - 16. [Appendix: working flows](#)
 - 17. [Troubleshooting](#)
-

1. Overview

TradeVision AI is a local, rule-based algorithmic trading and research console. It ingests market data, runs technical-analysis and pattern-detection pipelines, evaluates rule-based signals, produces recommendations with an AI explanation, and maintains a paper-trading dashboard with positions, orders, trades, PnL, and risk decisions.

The application is composed of:

- A **React + Vite frontend** (`frontend/`) served on port 5173 in development, with an authenticated SPA shell and a sidebar navigation.
- A **Django REST Framework + Django Channels backend** (`backend/`) running under **Daphne** (ASGI), served via **nginx** on port 80 in the local stack.
- A **Celery** worker/beat layer with multiple queues for background processing.
- **PostgreSQL** (TimescaleDB) as the data tier, fronted by **PgBouncer**.
- **Redis** for caching, pub/sub for channels, and Celery brokering.
- **Flower** for Celery monitoring and **Adminer** for ad-hoc database access.

The UI is a single-page application: every route is client-side; protected routes require a valid session (JWT).

2. Getting Started

2.1 What runs locally

Bring the stack up with:

```
docker compose \
  -f infra/docker-compose.yml \
  -f infra/docker-compose.dev.yml \
  up --build
```

The dev compose overlay mounts the `backend/` and `frontend/` sources live into their containers, so edits to code, Tailwind classes, Django views, and migrations are reflected on reload/restart without a full rebuild. The backend image is rebuilt only when requirements change.

Tip: If you recently edited `backend/config/urls.py` or static assets, restart the relevant container: `docker compose ... restart backend` or `... restart frontend`.

2.2 Ports & services

Service	Port	Notes
Frontend (Vite)	5173	Main UI. Relative API base <code>/api/v1</code> is proxied to the backend by Vite in dev.
nginx	80	Alternate entry point; proxies <code>/api/</code> to backend and <code>/</code> to frontend.
Backend (Daphne)	8000	Django ASGI. Direct backend access works; use <code>/api/v1/...</code> .
PgBouncer	6432	Connection pool in front of PostgreSQL (<code>transaction</code> pool mode, SCRAM-SHA-256).
PostgreSQL	5432	TimescaleDB pg16 (internal; reached via PgBouncer).
Redis	6379	

Service	Port	Notes
		Cache (db 1), channels (db 2), Celery broker/result (db 0).
Celery workers	—	<code>celery-worker-default</code> , <code>celery-worker-ai</code> , <code>celery-worker-market</code> ; see Appendix B for queue mapping.
Celery Beat	—	Periodic task scheduler.
Flower	<code>5575</code>	Celery monitoring UI (<code>admin / tradevision</code>).
Adminer	<code>8080</code>	Web DB client (pre-filled credentials).

2.3 Credentials

Target	User	Password	Notes
Postgres / PgBouncer / Adminer	<code>tradevision</code>	<code>tradevision_password</code>	Dev DB: <code>tradevision_dev_db</code>
Flower	<code>admin</code>	<code>tradevision</code>	
Web login	<code>admin</code>	<code>changeme123</code>	Created by <code>manage.py seed_admin</code> (username <code>admin</code> , email <code>admin@tradevision</code>)
Redis	—	—	No auth in dev.

Adminer is preconfigured by the container's `ADMINER_DEFAULT_*` environment variables, so on `http://localhost:8080` the server/user/password/database are filled in — just click **Login**.

2.4 First login

1. Open `http://localhost:5173` (or `http://localhost` through nginx).
 2. Navigate to **Login** (or you're redirected there if not authenticated).
 3. Enter **username** `admin` and **password** `changeme123`.
 4. On success, the browser stores an **access token** in memory and a **refresh token** in `localStorage`; `/auth/me/` is fetched to hydrate your profile, then you land on the **Dashboard -> Home**.
 5. The UI auto-refreshes the access token silently; on hard logout or token expiry, you're returned to `/login`.
-

3. Navigation structure

The authenticated UI is wrapped in `AppShell` — a left **Sidebar** + top **TopBar** + content area. The sidebar is organized into sections that mirror the backend services:

Dashboard - Home `/` · Portfolio `/portfolio` · Portfolio Summary `/portfolio/summary` · Positions `/positions` (detail `/positions/:id`) · Orders `/orders` (detail `/orders/:id`) · Trades `/trades` · Open Trades `/trades/open` · Closed Trades `/trades/closed` · Trade export status `/trades/export/:id`

Analytics & Risk - Analytics `/analytics` (-> `/analytics/:accountId/pnl`) · Risk Decisions `/risk` · Reconciliation `/reconciliation`

Research - Backtests `/research/backtests` (detail `/research/backtests/:id`) · Walk-Forward `/research/walk-forward` · Edge Validation `/research/edge-validation` · Cost Sensitivity `/research/cost-sensitivity` · News `/news`

Rules & Signals - Rule Configs `/rules` (detail `/rules/:ruleId`) · Rule Executions `/rules/executions` · Signals `/signals` · Recommendations `/recommendations` · Patterns `/patterns`

Memory & Watchlist - Trader Memory `/memory` · Watchlist `/watchlist`

System (read-only observability) - Journal `/journal` (detail `/journal/:correlationId`) · Audit `/audit` · Kill Switch `/risk/kill-switch` · Pipeline Health `/health` · Reconciliation `/reconciliation` · Raw Events `/system/ingestion`

At the far right of the **TopBar** is the **dark/light theme toggle** (Sun/Moon icon). Unauthenticated navigation falls back to `/login`.

Note: some System pages (`/audit`, `/system/ingestion`, `/reconciliation`) require API-key scopes (see Section 4.3) rather than just a logged-in session. A `viewer` - role user without the scope sees a **403** with a structured error banner rather than data.

4. Authentication & user management

Authentication is JWT-based. The first call exchanges `username` + `password` for an access (15 min) and refresh (7 days) token pair with rotation; the refresh token is rotated on every use and a 401 on a data request triggers a single silent refresh before failing.

4.1 Logging in

On the **Login** page, enter your username and password and click **Sign in**. - Success -> tokens stored -> redirect to the previous URL (or `/`). - `invalid_credentials` -> inline error: "Invalid

username or password." - `account_disabled` -> inline error describing the account is disabled.

Click the **Log out** (door) icon in the TopBar at any time to end the session.

4.2 Password & session

- The access token lives only in **memory** (never in `localStorage`); only the **refresh token** is persisted.
- A session expires (returns to `/login`) after the refresh token is revoked/expired or you click **Log out**.
- To reset a forgotten password, ask an **owner/staff** user to rotate it via `manage.py` against the Django auth store (there is no public "forgot password" flow).

4.3 API keys

There is no self-service page in the current UI; API keys are managed via the REST API (and Django admin / Adminer in dev).

- GET `/api/v1/auth/api-keys/` — list your keys.
- POST `/api/v1/auth/api-keys/` — create a key with a set of `scopes`.
- DELETE `/api/v1/auth/api-keys/<id>/` — revoke a key.

Use API keys for programmatic access to **scope-gated** endpoints (Journal, Audit, Raw Events, Reconciliation, Risk Decisions, etc.). Send them as `Authorization: Api-key <raw_key>`. The raw key is shown **once** at creation.

4.4 Roles & permissions

There are three user roles (set on the `User` model):

Role	Can
OWNER	Read/write across all data; manage users/keys; operate the kill switch.
STAFF	Read all data; operate the kill switch.
VIEWER (default)	Read-only on most endpoints; scope-gated routes return 403 without a qualifying key.

Scope-gated endpoints check the API key's scopes (e.g. `DASHBOARD_READ_RISK`). A `viewer` session (JWT) can browse the dashboard, positions, and charts but cannot write, cannot operate the kill switch, and cannot access scope-gated read endpoints without a qualifying API key.

5. Dashboard

The Dashboard is the authenticated landing zone.

`DashboardHome` (`/ / Home`) calls `GET /api/v1/dashboard/home/summary/` and renders a live trading-core snapshot: open positions, open orders, today's realized/unrealized PnL, active alerts, broker connection status, and market-session status.

5.1 Home

Quick-link tiles navigate to Portfolio, Trades, Rule Configs, Research, Audit, and Pipeline Health. StatCards surface open positions/orders, today's realized and unrealized PnL, active alerts, and broker connection status.

If you see "Account summary not initialized yet" (4):** this is the designed empty state — no projection data has been generated yet for the account. Run ingestion/ETL (or a backtest) so the dashboard read model is populated, then refresh.

5.2 Portfolio

GET `/dashboard/portfolio/composition/` — a donut of allocation percent by symbol plus three **StatCards**: Total market value, Total cost basis, Cash balance. Click any holding to drill in.

5.3 Positions

GET `/dashboard/positions/live/` — a paginated **DataTable** (sorted by symbol) of open positions with side (LONG/SHORT chip), quantity, avg cost, market value, unrealized P&L (color by sign), and opened-at timestamp. Click a row to open `/positions/:id`.

5.4 Orders

GET `/dashboard/orders/` — paginated table of orders: symbol, side, qty, entry/avg-fill price, filled quantity, status (FILLED/REJECTED/other), and created time. Click a row for `/orders/:id`.

5.5 Trades

`/trades` shows **Trades** (all), `/trades/open` shows Open Trades, and `/trades/closed` shows Closed Trades. These read from GET `/dashboard/trades/open/`, `/closed/`, and `/history/` respectively. Each table is sortable; clicking a row opens `/trades/:id`. A closed trade renders finalized outcome (won/lost/breakeven) in the **Journal**.

6. Analytics & Risk

6.1 PnL analytics

Analytics pages are account-scoped, reached via `/analytics` (which redirects to `/analytics/:accountId/pnl`). If `/auth/me/` does not expose a `default_account_id`, the Analytics index shows a notice telling you to open `/analytics/<accountId>/pnl` directly, or pick a backtest run from Research.

PnL Analytics renders StatCards (Current total PnL, Current unrealized PnL, Peak cumulative PnL, Current drawdown) plus a raw JSON time-series viewer (`GET /dashboard/accounts/:accountId/pnl`).

6.2 Analytics index redirect

`/analytics` is a convenience redirect, not a data page. It depends on the user's `default_account_id` from `/auth/me/`.

7. Risk Decisions

`/risk` -> `GET /api/v1/risk-management/decisions/` (paginated). A read-only list of persisted risk decisions: rule id, symbol, decision (APPROVED/REJECTED), severity, reason, and time. There is a visible warning "Contract not verified" notice because the response body shape was not formally audited.

8. Research / Backtesting

8.1 Backtests

`/research/backtests` — note the info alert that **no GET / backtesting/runs/ list endpoint** is documented. The page shows a **Create new backtest** form (`POST /backtesting/runs/`) and a "**Recently viewed**" panel that is tracked in `localStorage` (persists across sessions; removable with the trash button).

Creating a backtest opens it at `/research/backtests/:id` , which polls status until the run completes.

8.2 Edge Validation

`/research/edge-validation` — evaluates whether a strategy survives the full range of cost assumptions; classifications are `BREAKEVEN_FOUND` , `NEVER_PROFITABLE` , or `SURVIVES_FULL_RANGE` .

8.3 Cost Sensitivity

`/research/cost-sensitivity` — scans breakeven transaction-cost levels; color-coded chips (amber/emerald/rose).

8.4 Walk-Forward

`/research/walk-forward` — rolling-window analysis; deterministic via a `seed` parameter for reproducibility (documented in `docs/EDGE_VALIDATION_REPORT.md`).

9. Rules & Signals

9.1 Rule Configs

`/rules` -> `GET /rule-engine/configs/` . Columns: Rule ID, Enabled (chip), Severity override, ADR-029 gate (regimes, currently rendered as dash — validated regimes are **not** exposed by the configs API), and parameters JSON. It is **read-only** (no write contract exists). Click a row for `/rules/:ruleId` .

9.2 Rule Executions

`/rules/executions` -> `GET /rule-engine/executions/` (paginated), showing recent rule firings with their context.

9.3 Signals

`/signals` -> `GET /signals/` (paginated). Columns: ID, Symbol, Rule, Side (LONG/SHORT chip), Status (chip), and Created. Contains the same "contract not verified" notice.

10. Recommendations

`/recommendations` -> `GET /recommendations/` (paginated). Shows symbol, rule, side, status (PENDING/ACCEPTED/REJECTED), created time, and **inline Accept / Reject buttons** for pending recommendations. Accept/Reject open a **confirmation modal** before calling `POST /recommendations/:id/accept/` or `.../reject/` . The row updates in place (optimistic) and the AI explanation is available via `GET /recommendations/:id/explanation/` .

11. Memory & Watchlist

11.1 Trader Memory

`/memory` — two panels:

1. **Strategy projection:** enter a **Strategy ID** and click *Fetch projection* -> `GET /trader-memory/projections/:strategyId/`. Metrics render as collapsible JSON.
2. **Memory entries:** `GET /trader-memory/entries/` (paginated) — strategy, symbol, notes, created time.

11.2 Watchlist

`/watchlist` -> `GET /watchlist/?account_id=...`. Requires the `account_id` query parameter (the page notes this when missing). Columns: instrument token, trading symbol, exchange, name, segment, type, added time. Rows can be **removed** via the Delete button (`DELETE /watchlist/:instrument_token/`). PATCH is deferred (documented).

12. System & Observability

12.1 Journal

`/journal` (list) and `/journal/:correlationId` (detail). `GET /journal/entries/?account_id=...`. A read-only audit-quality trade journal with decision snapshots, outcomes, and realized PnL.

12.2 Audit Log

`/audit` -> `GET /audit/entries/` (paginated). Columns: ID, Actor, Action, Target type, Target ID, Time. Read-only, authoritative record of system actions.

12.3 Pipeline Health

`/health` -> `GET /pipeline-health/` plus the unauthenticated `/health/<name>/` component checks (`db` , `cache` , `celery` , `eventbus` , `system`). Shows a per-component status breakdown and a pipeline status summary (status, last run, staleness in seconds, components).

12.4 Kill Switch

`/risk/kill-switch` -> `GET /risk-management/kill-switch/` + activate/deactivate. Shows current state (ACTIVE/inactive) and a history of activations. Only **owner** or **staff** roles can toggle; viewers see a permission notice.

12.5 Reconciliation

`/reconciliation` -> `GET /portfolio-reconciliation/drift/summary/` (read-only drift report). Owner/Staff-only; requires the appropriate API-key scope for a viewer.

12.6 Raw Events

`/system/ingestion` -> `GET /ingestion/raw-events/` (paginated). Staff-only raw event listing for ingestion debugging.

12.7 News

`/news` -> `GET /news/` (paginated, optional `?symbol=` filter). Columns: Source, Headline (links out to the provider), Symbols,

Sentiment score (colored chip: ≥ 0.25 green, ≤ -0.25 red, otherwise neutral).

12.8 Patterns

`/patterns` -> GET `/pattern-engine/runs/` (paginated), listing pattern-detection runs with status and metadata.

13. UI features

13.1 Dark / light mode

A **Sun/Moon toggle** is in the TopBar next to the logout button. - It writes the choice to `localStorage` (`tv-theme`); your preference persists across reloads. - If no preference is stored, the app follows your OS (`prefers-color-scheme: dark`). - Toggling adds/removes the `dark` class on `<html>` ; converted surfaces restyle (sidebar, cards, tables, charts, inputs, focus rings, scrollbars). - Charts switch their grid/tick/tooltip colors via CSS variables, so series colors remain intact while the canvas background inverts.

13.2 Data states

Every fetch-driven page uses the same small set of states (`useFetch`): - **loading** -> skeleton rows/spinners. - **error** -> an `Alert` banner with the error code and a **Retry** button. - **empty** -> an `EmptyState` with a title/description. - **success** -> real content; list pages also show "Showing X of Y" when results are truncated by pagination. - **401 on a data call** -> a single silent token refresh is attempted; if that also fails you're redirected to `/login` .

14. Configuration reference

14.1 Environment variables (compose)

These are defined in `infra/.env` and consumed by `infra/docker-compose.yml` / `infra/docker-compose.dev.yml`.

Variable	Default
<code>DJANGO_SETTINGS_MODULE</code>	<code>config.settings.development</code>
<code>DJANGO_SECRET_KEY</code>	<code>change-me-to-a-long-random-string-in-production</code>
<code>DATABASE_URL</code>	<code>postgresql://...@pgbouncer:6432/tradevision_dev_db</code>
<code>POSTGRES_DB</code> / <code>_USER</code> / <code>_PASSWORD</code> / <code>_HOST</code> / <code>_PORT</code>	dev defaults
<code>POSTGRES_HOST</code>	<code>pgbouncer</code> / port <code>6432</code>
<code>REDIS_URL</code> / <code>REDIS_CACHE_URL</code> / <code>REDIS_CHANNELS_URL</code>	<code>redis://redis:6379/{0,1,2}</code>
<code>CELERY_TASK_ALWAYS_EAGER</code>	<code>False</code>

Variable	Default
ADMIN_EMAIL / ADMIN_PASSWORD	admin@tradevision.ai / changeme123
AI_PROVIDER	gemini
GEMINI_API_KEY , OPENAI_API_KEY , ANTHROPIC_API_KEY , OLLAMA_BASE_URL , DEEPSEEK_API_KEY	—
AI_CONFIDENCE_FLOOR , AI_DAILY_BUDGET_USD , AI_DEDUP_WINDOW_SECONDS , AI_MAX_TOKENS	0.55 / 10.00 / 300 / 4096
CIRCUIT_BREAKER_FAILURE_THRESHOLD / _RECOVERY_TIMEOUT / _EXPECTED_EXCEPTION	5 / 60 / Exception
MARKET_DATA_POLL_*	(in file)
NEWS_API_KEY / NEWS_API_BASE_URL	—

The frontend reads VITE_API_BASE_URL (default relative /api/v1). In the dev overlay it is empty, so Vite proxies /api and /ws to the backend.

14.2 Django settings overrides

Per-environment settings live in `backend/config/settings/` :
`base.py` (shared), `development.py` (`DEBUG=True` , debug toolbar, browsable API renderer, `ALLOWED_HOSTS=["*"]`),
`dev.py` , `staging.py` , `testing.py` , `test.py` ,
`production.py` . The local dev stack uses `development` .

Key shared knobs: `PAGE_SIZE=20` (global pagination), JWT lifetimes (access 15 min / refresh 7 days, both rotated), Celery queue->task routing (see Appendix B), and throttle rates (auth endpoint `15/min`).

15. API reference (summary)

All endpoints are rooted at `/api/v1/` (and `/__debug__` / dev toolbar, `/metrics`). JWT Bearer on protected routes; API-key scopes on gated routes. List endpoints honor global pagination (`page` , `page_size`); default page size is 20.

Method	Path	Auth	Purpose
POST	<code>/auth/login/</code>	anon	Issue {access, refresh, user} .
POST	<code>/auth/refresh/</code>	anon	Rotate access token.
GET	<code>/auth/me/</code>	JWT	User profile (id, username, role, date_joined).
GET	<code>/auth/api-keys/</code>	JWT	List own API keys.
POST	<code>/auth/api-keys/</code>	JWT	Create a key (scopes).

Method	Path	Auth	Purpose
DELETE	/auth/api-keys/<id>/	JWT	Revoke a key.
GET	/health/	anon	Liveness probe + component checks (db/cache/ celery/ eventbus/ system).
GET	/dashboard/home/summary/	JWT	Dashboard snapshot.
GET	/dashboard/portfolio/composition/	JWT	Portfolio holdings/ allocation.
GET	/dashboard/positions/live/	JWT	Open positions (paginated).
GET	/dashboard/orders/	JWT	Orders (paginated).
GET	/dashboard/trades/open/	JWT	Open trades (paginated).
GET	/dashboard/trades/closed/	JWT	Closed trades (paginated).
GET	/dashboard/trades/history/	JWT	Trade history (paginated).
GET	/dashboard/accounts/:accountId/pnl	JWT	PnL analytics time series.
GET	/accounts/	JWT	Accounts (M4).
GET	/rule-engine/configs/	JWT	Rule configs (read-only).
GET		JWT	Single rule config.

Method	Path	Auth	Purpose
	/rule-engine/configs/<rule_id>/		
GET	/rule-engine/executions/	JWT	Rule executions (paginated).
GET	/signals/	JWT	Signals (paginated).
GET	/recommendations/	JWT	Recommendations (paginated).
POST	/recommendations/:id/accept/	JWT	Accept a recommendation.
POST	/recommendations/:id/reject/	JWT	Reject a recommendation.
GET	/recommendations/:id/explanation/	JWT	AI explanation text.
GET	/trader-memory/entries/	JWT	Memory entries (paginated).
GET	/trader-memory/projections/:strategyId/	JWT	Strategy projection metrics.
GET	/watchlist/?account_id=...	JWT	Watchlist for an account (paginated).
DELETE	/watchlist/:instrument_token/	JWT	Remove a watchlist instrument.
GET	/journal/entries/?account_id=...	scope	Journal entries (paginated).
GET	/journal/entries/:id/	scope	

Method	Path	Auth	Purpose
			Journal entry detail.
GET	/audit/entries/	scope	Audit log (paginated).
GET	/pipeline-health/	JWT	Pipeline status.
GET	/risk-management/decisions/	scope	Risk decisions (paginated).
GET	/risk-management/kill-switch/	JWT	Kill switch state.
POST	/risk-management/kill-switch/activate/	OWNER/ STAFF	Activate.
POST	/risk-management/kill-switch/deactivate/	OWNER/ STAFF	Deactivate.
GET	/portfolio/	JWT	Portfolio (M4).
GET	/portfolio/positions/	JWT	Positions (M4).
GET	/execution/orders/	JWT	Orders (B).
GET	/execution/orders/<id>/	JWT	Order detail (B).
GET	/execution/positions/	JWT	Execution positions (B).
GET	/portfolio-reconciliation/drift/summary/	OWNER/ STAFF	Drift summary (read-only).
GET	/portfolio-reconciliation/drift/	OWNER/ STAFF	Drift entries (paginated).
GET	/ingestion/raw-events/	scope/ staff	Raw ingestion events (paginated).

Method	Path	Auth	Purpose
POST	/ingestion/webhook/	anon	External webhook entry point.
GET	/news/	JWT	News headlines (paginated).
GET	/pattern-engine/runs/	JWT	Pattern runs (paginated).
POST	/backtesting/runs/	JWT	Create a backtest run.
GET	/metrics/	anon	Prometheus metrics.

anon = no auth required (health/metrics/public webhooks).

Appendix A: Working flows

Login -> Dashboard. Open /login , enter credentials. The app POSTs to /auth/login/ , stores the access token in memory and refresh token in localStorage , fetches /auth/me/ , and loads the Dashboard Home summary. Every subsequent /api/v1/... call attaches Authorization: Bearer <access>; on 401 it refreshes once, then redirects to /login if still failing.

Reviewing a recommendation. Navigate to Recommendations (/recommendations). For each PENDING item, click Accept or Reject; a confirmation modal appears. Confirm -> POST to :id/accept/:id/reject . The row updates in place (optimistic). The explanation is available via :id/explanation/ .

Running a backtest. Research -> Backtests (/research/backtests). Submit the create form (POST /backtesting/runs/). The new run appears in Recently viewed; open /

`research/backtests/:id` to poll status. While running, results are partial; on completion the run shows final metrics.

Checking pipeline health. System -> Pipeline Health (`/health`). The page fires the unauthenticated `/api/v1/health/` root check plus `/health/db|cache|celery|eventbus|system/`, and reads `/pipeline-health/` for staleness counters and component statuses. A red chip means that stage hasn't reported within its expected window.

Operating the kill switch. System -> Kill Switch (`/risk/kill-switch`). Only OWNER/STAFF see actionable buttons. Activate halts trading; deactivate resumes. History (who/when/why) is listed below current state.

Viewing API keys. There's no UI page yet; manage keys via API: `GET /auth/api-keys/`, `POST /auth/api-keys/` with `{"scopes":["..."]}`, `DELETE /auth/api-keys/<id>/`. The raw key is returned only at creation.

Theme toggle. Click the Sun/Moon icon in the TopBar. Choice is remembered; system default is used on first visit.

Appendix B: Celery queue -> task mapping

Queue	Tasks
decisions	execution <code>handle_risk_approved</code> ; backtesting <code>run_backtest</code> ; pattern_engine runs (analytics); market_data poll
execution	execution <code>process_order</code>
ai_reasoning	

Queue	Tasks
	strategy_registry match; ai_engine evaluate_confidence; recommendations create compose_explanation
rule_engine	rule_engine evaluate_packet; publish_rule_firing
analytics	journal finalize_stale_entries; pattern_engine precompute; trader_memory record/rebuild
maintenance	eventbus dispatch/polling/replay; accounts purge_expired_tokens; pipeline_health evaluate
market_data	(poll consumer)

Appendix C: Troubleshooting

Q: I log in but `/analytics` shows no default account. A: `/auth/me/` doesn't expose `default_account_id`. Open `/analytics/<accountId>/pnl` directly.

Q: A page shows a red error banner with a code. A: The error is from the structured exception handler. Common codes: `invalid_credentials`, `account_disabled`, `invalid_refresh_token`, `account-summary-not-initialized` (404, no projection data yet). Click Retry.

Q: `/watchlist` returns 400. A: The watchlist API requires `?account_id=...`.

Q: Audit, Raw Events, Reconciliation return 403. A: These require an API key with a specific scope, not just a JWT session. Create a suitably-scoped key.

Q: Dark mode toggle doesn't apply. A: Ensure you're served from `localhost:5173` ; the class is applied to `<html>` . Hard-refresh if needed.

Q: Frontend API calls 404 (network error) from port 5173. A: Vite must proxy `/api` to the backend. If you edited `vite.config.ts` and stale `vite.config.js` is present, delete `frontend/vite.config.js` and `frontend/vite.config.d.ts` and restart the frontend container.

Q: Login returns HTTP 500. A: With `DEBUG=True` , ensure `config/urls.py` includes `debug_toolbar.urls` under `__debug__ /` (it does by default in dev settings).

Q: "relation dashboard_homesummary does not exist" (500 on dashboard). A: Run `docker compose ... exec backend python manage.py migrate` to apply the dashboard migrations.

End of manual.